

电商微服务开发手册

xxx项目需求分析总体模块设计流程描述登陆流程订单状态

布置的作业题目是：

使用SpringCloud完成Workshop

1)实现用户浏览并购买商品服务

创建

2)实现根据用户购买的商品进行其它商品的推荐服务

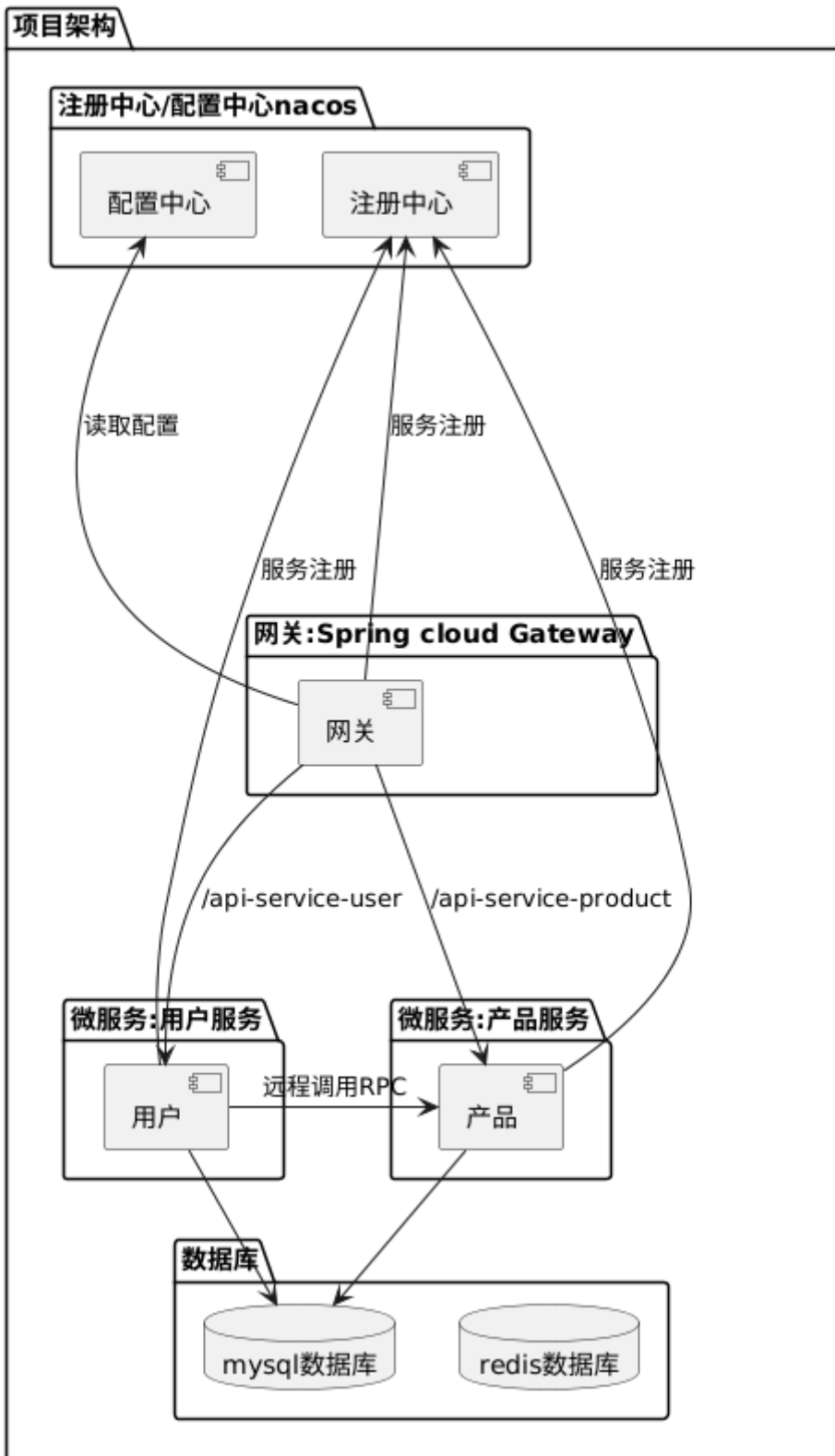
3)实现两个服务之间的RPC调用

项目架构

代码块

```
1  package "项目架构" {
2
3
4  package "网关:Spring cloud Gateway" {
5  component "网关" as gateway
6  }
7
8
9  package "注册中心/配置中心nacos" {
10 component 注册中心
11 component 配置中心
12 }
13
14
15 package "微服务:用户服务" {
16 component 用户
17
18 }
19
20 package "微服务:产品服务" {
21 component 产品
```

```
22
23 }
24
25
26
27 [用户] --up-> [注册中心] :服务注册
28 [产品] ---up-> [注册中心] :服务注册
29 [gateway] ---up-> [注册中心] :服务注册
30 [gateway] ---up-> [配置中心] :读取配置
31
32
33 [gateway] --down-> [用户] :/api-service-user
34 [gateway] --down-> [产品] :/api-service-product
35
36 package "数据库"{
37     database "mysql数据库"
38     database "redis数据库"
39
40 }
41
42 [用户] --> "mysql数据库"
43 [产品] --> "mysql数据库"
44
45 [用户] --right-> [产品] :远程调用RPC
46
47 }
```



总体模块设计

代码块

```
1  (org.example.clouddemo)
2  |--common
3  |--gateway
4  |--service
5      |--model
6      |--service-recommend
```

```
7      |--service-order
8      |--feign:http配置
9      |--order
10     |--controller:接口管理
11     |service.common
12     |--impl:功能实现类
13     |--mapper:数据库映射
14     |--param:http参数包装对象
15 |--service-product
16 |--feign:http配置
17 |--product
18 |--controller:接口管理
19 |service.common
20 |--impl:功能实现类
21 |--mapper:数据库映射
22 |--param:http参数包装对象
23 |--service-user
```

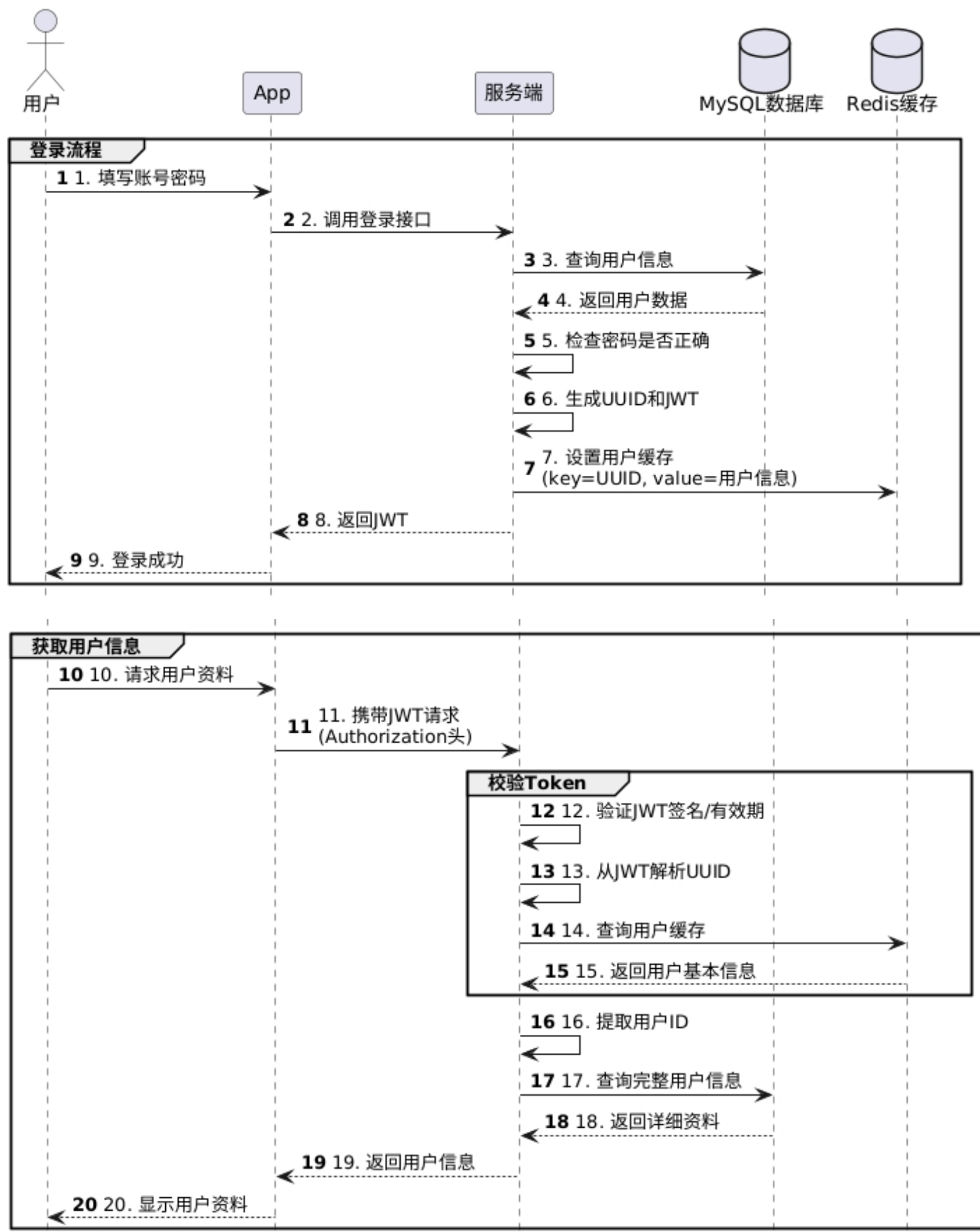
流程描述

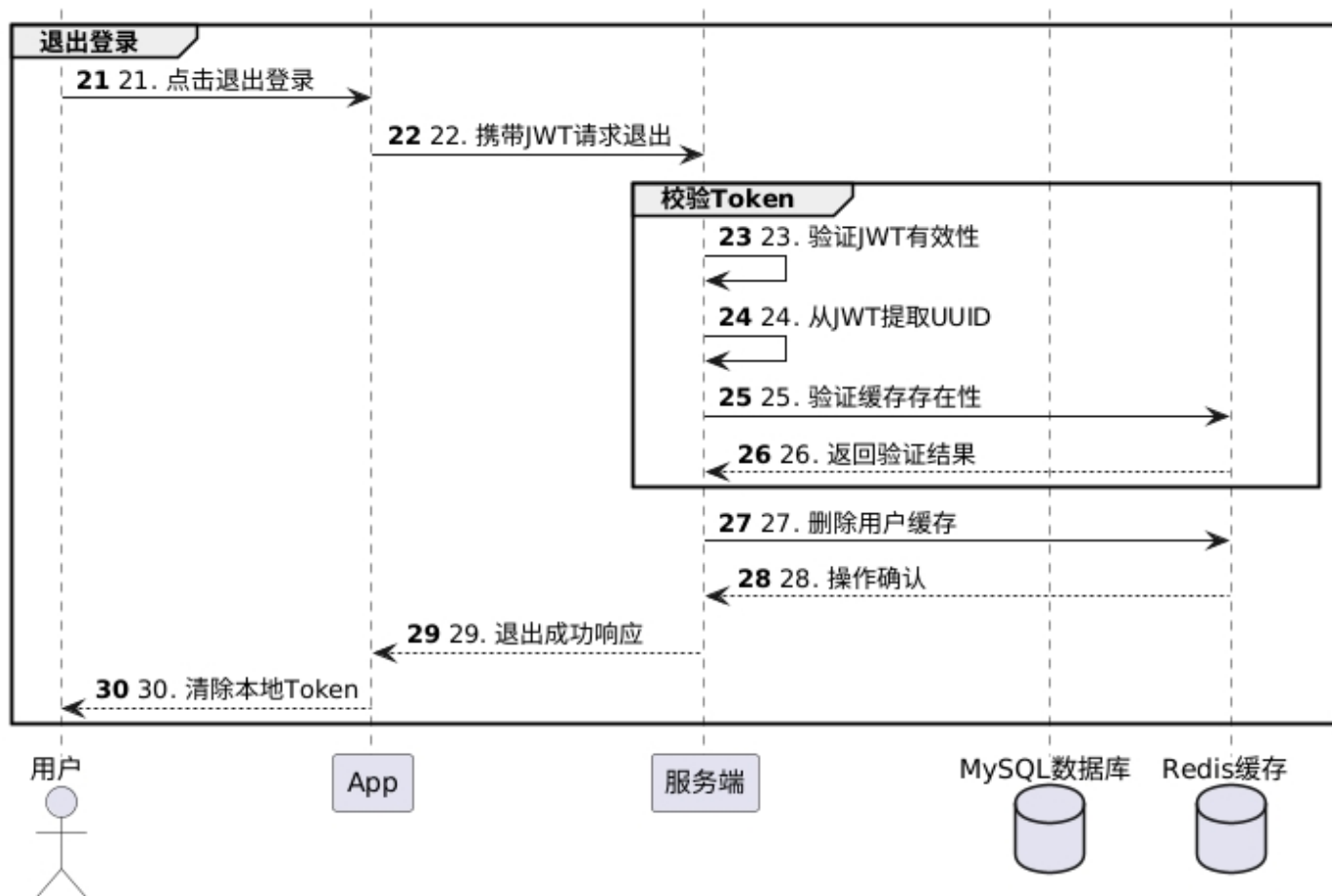
登陆流程

代码块

```
1  @startuml
2
3  actor 用户 as user
4  participant App as app
5  participant 服务端 as server
6  database MySQL数据库 as mysql
7  database Redis缓存 as redis
8
9  autonumber
10
11 group 登录流程
12 user -> app : 1. 填写账号密码
13 app -> server : 2. 调用登录接口
14 server -> mysql : 3. 查询用户信息
15 mysql --> server : 4. 返回用户数据
16 server -> server : 5. 检查密码是否正确
17 server -> server : 6. 生成UUID和JWT
18 server -> redis : 7. 设置用户缓存\n(key=UUID, value=用户信息)
19 server --> app : 8. 返回JWT
20 app --> user : 9. 登录成功
```

```
21  end
22
23  group 获取用户信息
24  user -> app : 10. 请求用户资料
25  app -> server : 11. 携带JWT请求\n(Authorization头)
26  group 校验Token
27  server -> server : 12. 验证JWT签名/有效期
28  server -> server : 13. 从JWT解析UUID
29  server -> redis : 14. 查询用户缓存
30  redis --> server : 15. 返回用户基本信息
31  end
32  server -> server : 16. 提取用户ID
33  server -> mysql : 17. 查询完整用户信息
34  mysql --> server : 18. 返回详细资料
35  server --> app : 19. 返回用户信息
36  app --> user : 20. 显示用户资料
37  end
38
39  group 退出登录
40  user -> app : 21. 点击退出登录
41  app -> server : 22. 携带JWT请求退出
42  group 校验Token
43  server -> server : 23. 验证JWT有效性
44  server -> server : 24. 从JWT提取UUID
45  server -> redis : 25. 验证缓存存在性
46  redis --> server : 26. 返回验证结果
47  end
48  server -> redis : 27. 删除用户缓存
49  redis --> server : 28. 操作确认
50  server --> app : 29. 退出成功响应
51  app --> user : 30. 清除本地Token
52  end
53
54  @enduml
```





订单状态

代码块

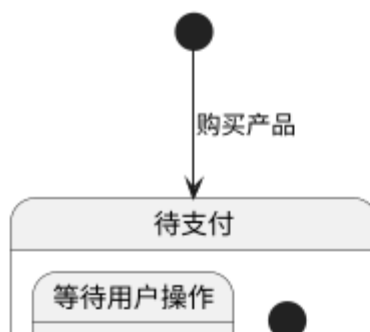
```

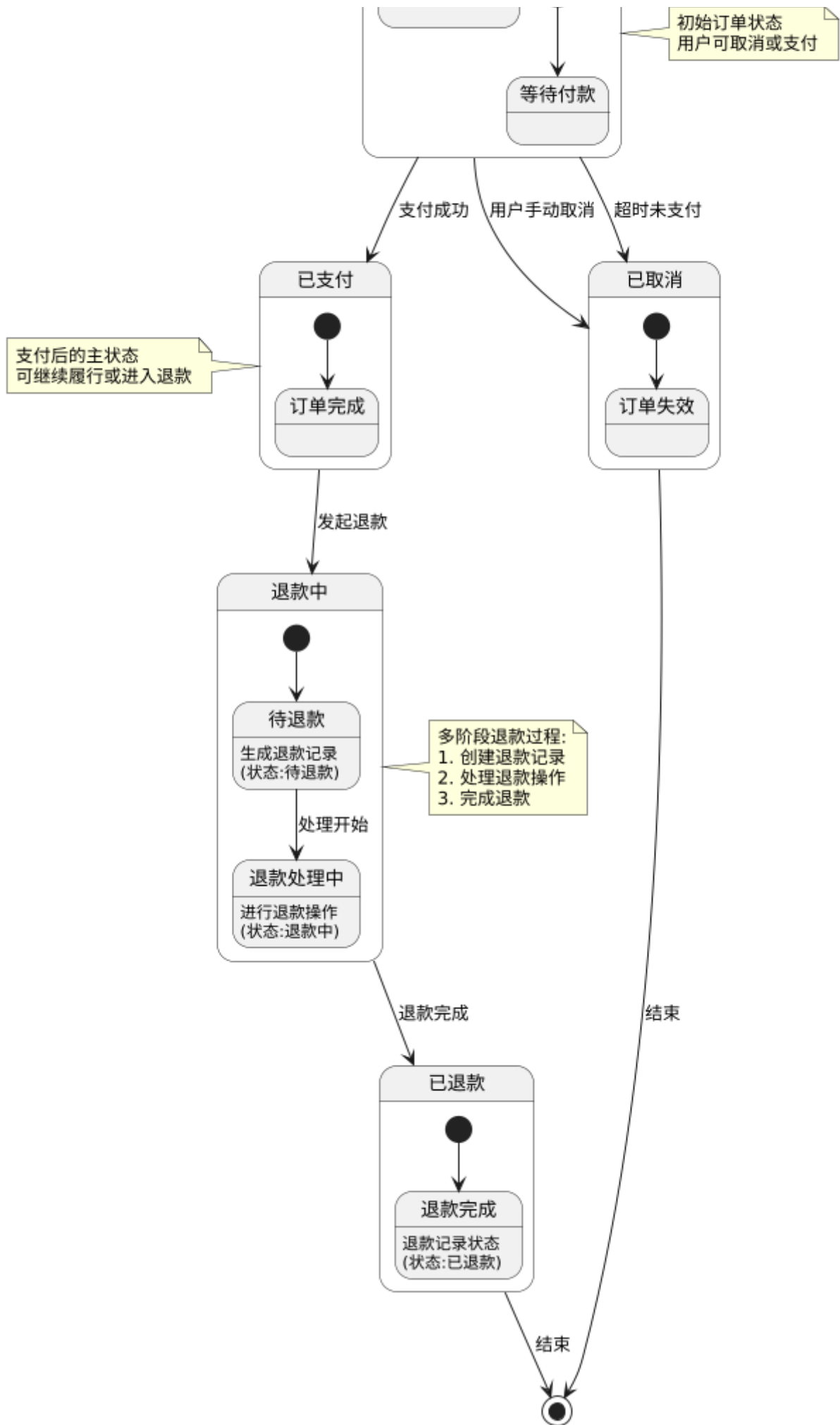
1  @startuml
2  scale 600 width
3
4  [*] --> 待支付 : 购买产品
5
6  state 待支付 {
7      [*] --> 等待付款
8      state "等待用户操作" as wait
9  }
10
11 state 已支付 {
12     [*] --> 订单完成
13 }
14
15 state 退款中 {
16     [*] --> 待退款
17     待退款 : 生成退款记录\n(状态:待退款)
18     待退款 --> 退款处理中 : 处理开始
19     退款处理中 : 进行退款操作\n(状态:退款中)
  
```

```

20 }
21
22 state 已退款 {
23     [*] --> 退款完成
24     退款完成 : 退款记录状态\n(状态:已退款)
25 }
26
27 state 已取消 {
28     [*] --> 订单失效
29 }
30
31 待支付 --> 已取消 : 用户手动取消
32 待支付 --> 已取消 : 超时未支付
33 待支付 --> 已支付 : 支付成功
34
35 已支付 --> 退款中 : 发起退款
36 退款中 --> 已退款 : 退款完成
37
38 已取消 --> [*] : 结束
39 已退款 --> [*] : 结束
40
41 note right of 待支付
42     初始订单状态
43     用户可取消或支付
44 end note
45
46 note right of 退款中
47     多阶段退款过程:
48     1. 创建退款记录
49     2. 处理退款操作
50     3. 完成退款
51 end note
52
53 note left of 已支付
54     支付后的主状态
55     可继续履行或进入退款
56 end note
57 @enduml

```





订单模块

- 添加订单
- 删除订单
- 修改订单
- 查看订单

推荐模块

- 1.根据一个用户的过往订单归类次数倒序推荐商品
- 2.如果过往订单没有的话就推荐价格最贵的前三个
- 3.只需要查订单表操作，自动调用：登录显示推荐

数据库设计

数据库名: clouddemo

数据表规则:

- 主键 `id` 尽量使用 `bigint(20)`,使用雪花算法生成自增的long
- 创建时间`create_time`,修改时间`update_time`
- 排序字段使用`sort_index`
- 表示是否xx的字段,使用`xx_flag`;例如是否删除,使用`del_flag`
- 是否删除: `del_flag`;是否启用:`enable_flag`

所有表默认有以下字段

字段	类型	唯一	名称	说明
<code>create_time</code>	<code>datetime</code>	-	创建时间	-
<code>update_time</code>	<code>datetime</code>	-	更新时间	-
数据库名称: <code>clouddemo</code>				

utf8mb4 utf8mb4_bin

用户表t_user

--	--	--	--	--	--	--

字段	类型	对应java类型	唯一	非空	名称	说明
id	bigint(20)	Long	是	是	id, 自增长,主键	-
username	varchar(50)	String	是	是	帐号,自动生成,9位数字	-
password	varchar(100)	String	-	是	密码,加密存储	默认为null
nickname	varchar(50)	String	-	-	昵称,可修改	默认为null
avatar_url	varchar(200)	String	-	-	头像,可修改	图片的base64数据，默认为null
enable_flag	tinyint(1)		-	是	是否启用	1:是 0:否，默认为1
money	decimal(10,1)		-	是	余额	默认为0.0

产品表t_product

字段	类型	唯一	非空	名称	说明
id	bitint(20)	是	是	id 主键	-
product_name	varchar(50)	是	是	产品名称	-
product_img	varchar(200)	-	-	产品图片	存储图片的base64数据，默认为null
product_desc	varchar(500)	-	-	产品描述	默认为null
price	decimal(10,2)	-	是	产品单价	-
product_num	int (1)	-	是	产品当前数量	默认为0
sort_index	int	-	-	排序字段	默认为 ‘ 0’ ’
enable_flag	tinyint(1)	-	是	是否启用	1:是 0:否，默认为1

订单表t_order

这里规定一条订单对应一个人购买的一种商品，不设置订单明细表

字段	类型	唯一	非空	名称	说明	描述
id	bigint(20)	是	是	订单id	主键	
user_id	bigint(20)	-	是	用户Id	-	索引User表
product_id	bigint(20)	-	是	商品Id	-	索引Product表
quantity	int (1)	-	是	数量	-	该类商品的数量
total_price	decimal (20)	-	是	订单总金额	-	该条订单的总金额
status	tinyint (1)	-	是	是否支付	-	1:是 0:否

接口文档

一、基础信息

- 网关地址: 服务所在主机IP:9027
- 用户服务路由前缀: /api-service-user
- 产品服务路由前缀: /api-service-product
- 订单服务路由前缀: /api-service-order
- 推荐服务路由前缀: /api-service-recommend
- 认证方式: Token认证（Header中传递 token 字段）
- 所有接口统一返回格式：

代码块

```
1  {  
2    "code": 200,           // 状态码, 200表示成功
```

```
3     "message": "操作成功", // 响应消息
4     "data": {}           // 响应数据
5 }
```

- 状态码说明

1. 200 : 操作成功
2. 400 : 参数错误
3. 401 : 未授权(token无效或过期)
4. 403 : 权限不足
5. 404 : 资源不存在
6. 500 : 服务器内部错误

- swagger文档

项目集成了Knife4j, 可以通过以下地址访问在线API文档:

1. 用户服务文档: `http://{service_user_ip}:{user-service-port}/doc.html`
2. 产品服务文档: `http://{service_product_ip}:{product-service-port}/doc.html`

二、用户服务接口 (service-user)

APP接口 (已测)

用户注册

- 接口地址: `POST /api-service-user/api/app/user/register`
- 接口描述: 用户注册, 无需token
- 请求参数:

代码块

```
1  {
2      "username": "gemo123",           // 用户名, 必填
3      "password": "111111"           // 密码, 必填
4  }
```

- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": "1937410285742366720", // 用户ID
4      "msg": "success",
5      "success": true
6  }
```

- 测试用例

代码块

```
1  curl -X POST -H "Content-Type: application/json" -d "{
    \"username\": \"testuser\", \"password\": \"123456\" }"
    http://10.24.3.163:9025/api-service-user/api/app/user/register
```

用户登录

- 接口地址: POST /api-service-user/api/app/user/login
- 接口描述: 用户登录, 无需token
- 请求参数:

代码块

```
1  {
2      "username": "root",
3      "password": "123456"
4  }
```

- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": {
4          "token":
            "eyJhbGciOiJIUzI1NiJ9.eyJqdGkiOiIwYjg1MDBhYS0wZmYzLTQxNTAtYWQ2YS1jYjIxNTE5MzkxMzMiLCJzZWIiOiJ7XCJpZFWiOjE5Mzc0MTAyODU3NDIzNjY3MjAsXCJsb2dpblR5cGVcIjpcInB3ZFwiLFwidXNlck5hbWVcIjpcImN3dFwiLFwidXNlclR5cGVcIjpcImFwcFwiLFwidXVpZFwiOlwiMGi4NTAwYWEtMGZmMy00MTUwLWFKNmEtY2I5MTUxOTM5MTMzXCJ9IiwiaWF0IjoxNzUwNzQ5ODU2LCJleHAiOjE3NTEwMDkwNTYsInJvbGVzIjoiaXNlciIsInR5cGUiOiJhcHAifQ.n1vW_C7FuwtmkfAumCKAj_uis-FC4h73inlmEN8-vvg"
5      },
6      "msg": "success",
7      "success": true
8  }
```

获取用户信息

- **接口地址:** POST /api-service-user/api/app/user/getInfo
- **接口描述:** 获取当前登录用户信息，需要token
- **请求头:** token: {登录时返回的token}
- **请求参数:** {}
- **响应示例:**

代码块

```
1  {
2      "code": 1,
3      "data": {
4          "id": "1937410285742366720",
5          "username": "cwt",
6          "nickname": "cwt",
7          "avatarUrl": null,
8          "enableFlag": true,
9          "money": 0.0,
10         "createTime": "2025-06-24 15:19:42",
11         "updateTime": "2025-06-24 15:19:42"
12     },
13     "msg": "success",
14     "success": true
15 }
```

更新用户信息

更新用户

- 接口地址: `POST /api-service-user/api/app/user/update`
- 接口描述: 更新用户信息
- 请求参数:

代码块

```
1  // 参数可选，与User对象一致
2  {
3      "id": "Long", // 用户ID 必须包含
4      "username": "String", // 帐号（自动生成，9位数字）
5      "password": "String", // 密码（加密存储）
6      "nickname": "String", // 昵称（可修改）
7      "avatarUrl": "String", // 头像URL（可修改）
8      "enableFlag": "Boolean", // 是否启用（1-是，0-否）
9      "money": "Double", // 用户余额
10     "createTime": "LocalDateTime", // 创建时间（格式：yyyy-MM-dd HH:mm:ss）
11     "updateTime": "LocalDateTime", // 更新时间（格式：yyyy-MM-dd HH:mm:ss）
12     "productNum": "Integer" // 库存数量
13 }
14
```

退出登录

- 接口地址: `POST /api-service-user/api/app/user/logout`
- 接口描述: 用户退出登录，需要token
- 请求头: `token: {登录获取的token}`
- 请求参数: `{}`
- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": 1,
4      "msg": "success",
5      "success": true
6  }
```


Admin接口

用户列表

- **接口地址:** POST /api-service-user/api/admin/user/list/{page}/{pageSize}
- **接口描述:** 分页获取用户列表
- **路径参数:**
 - `page`: 页码
 - `pageSize`: 每页数量
- **请求参数:**

代码块

```
1 // 参数可选, 与User对象一致
2 {
3     "id": "Long", // 用户ID
4     "username": "String", // 帐号
5     "password": "String", // 密码
6     "nickname": "String", // 昵称
7     "avatarUrl": "String", // 头像URL (可修
8     "enableFlag": "Boolean", // 是否启用 (1-是, 0-否)
9     "money": "Double", // 用户余额
10    "createTime": "LocalDateTime", // 创建时间 (格式: yyyy-MM-dd HH:mm:ss)
11    "updateTime": "LocalDateTime", // 更新时间 (格式: yyyy-MM-dd HH:mm:ss)
12    "productNum": "Integer" // 库存数量
13 }
14
```

- **响应结果**

代码块

```
1 {
2     "code": 200,
3     "message": "操作成功",
4     "data": {
5         "id": 1931333439791996928,
6         "username": "gemo123",
7         "nickname": "用户昵称",
8         "avatarUrl": "头像URL",
9         "enableFlag": true,
10        "createTime": "2025-01-06T10:30:00",
11        "updateTime": "2025-01-06T10:30:00"
```

```
12     }
13     ...
14 }
```

查询单个用户

- **接口地址:** POST /api-service-user/api/admin/user/queryOne
- **接口描述:** 根据条件查询单个用户
- **请求参数:**

代码块

```
1 // 参数可选, 与User对象一致
2 {
3     "id": "Long", // 用户ID 必须包含
4 }
5
```

- **响应结果:**

代码块

```
1 {
2     "code": 200,
3     "message": "操作成功",
4     "data": {
5         "id": 1931333439791996928,
6         "username": "gemo123",
7         "nickname": "用户昵称",
8         "avatarUrl": "头像URL",
9         "enableFlag": true,
10        "createTime": "2025-01-06T10:30:00",
11        "updateTime": "2025-01-06T10:30:00"
12    }
13 }
```

删除用户

- **接口地址:** POST /api-service-user/api/admin/user/delete/{id}
- **接口描述:** 删除指定用户

- 路径参数: `id` : 用户ID

更新用户

- 接口地址: `POST /api-service-user/api/admin/user/update`
- 接口描述: 更新用户信息
- 请求参数:

代码块

```
1 // 参数可选, 与User对象一致
2 {
3     "id": "Long", // 用户ID 必须包含
4     "username": "String", // 帐号 (自动生成, 9位数字)
5     "password": "String", // 密码 (加密存储)
6     "nickname": "String", // 昵称 (可修改)
7     "avatarUrl": "String", // 头像URL (可修改)
8     "enableFlag": "Boolean", // 是否启用 (1-是, 0-否)
9     "money": "Double", // 用户余额
10    "createTime": "LocalDateTime", // 创建时间 (格式: yyyy-MM-dd HH:mm:ss)
11    "updateTime": "LocalDateTime", // 更新时间 (格式: yyyy-MM-dd HH:mm:ss)
12    "productNum": "Integer" // 库存数量
13 }
14
```

三、产品服务接口 (service-product)

App接口 (已测)

创建产品

- 接口地址: `POST /api-service-product/api/app/product/create`
- 接口描述: 创建产品, 需要token
- 请求头: `token: {登录获取的token}`
- 请求参数:

代码块

```
1 {
2     "productName": "产品1", // 产品名称, 必填
3     "productImg": "http://x.png", // 产品图片, 可选
```

```
4     "price": 10.00,           // 价格, 必填
5     "productDesc": "产品描述", // 产品描述, 必填
6     "sortIndex": 0,           // 排序, 可选, 默认0
7     "enableFlag": true        // 是否启用, 可选, 默认true
8 }
```

- 请求示例:

代码块

```
1  {
2      "productName": "哈密瓜",
3      "productImg": "",
4      "price": "20",
5      "productDesc": "这是一个哈密瓜",
6      "sortIndex": "0",
7      "enableFlag": "true"
8  }
```

- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": 1,
4      "msg": "success",
5      "success": true
6  }
```

更新产品

- 接口地址: POST /api-service-product/api/app/product/update
- 接口描述: 更新产品信息, 需要token
- 请求头: token: {登录获取的token}
- 请求参数:

代码块

```
1  {
2      "id": "1936321866654285824", // 产品ID, 必填
3      "productName": "雪梨", // 产品名称, 必填
```

```
4    "productImg": "", // 产品图片, 可选
5    "productDesc": "这是雪梨", // 产品描述, 可选
6    "price": 30, // 价格, 可选
7    "num": 400, // 数量, 可选
8    "sortIndex": 0, // 排序, 可选
9    "enableFlag": true // 是否启用, 可选
10 }
```

- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": 1,
4      "msg": "success",
5      "success": true
6  }
```

删除产品

- 接口地址: `POST /api-service-product/api/app/product/delete/{id}`
- 接口描述: 删除产品, 需要token
- 请求头: `token: {登录获取的token}`
- 路径参数: `id` 产品ID
- 响应示例:

代码块

```
1  {
2      "code": 200,
3      "message": "操作成功",
4      "data": true
5  }
```

产品详情

- 接口地址: `POST /api-service-product/api/app/product/queryOne`
- 接口描述: 获取产品详情, 无需token

- 请求参数:

代码块

```
1  {
2    "id": 1931396915971227648 // 产品ID, 必填
3  }
```

- 响应示例:

代码块

```
1  {
2    "code": 1,
3    "data": {
4      "id": "1936321866654285824",
5      "productName": "雪梨",
6      "productImg": "",
7      "productDesc": "这是雪梨",
8      "price": 50,
9      "productNum": null,
10     "sortIndex": 0,
11     "enableFlag": true,
12     "createTime": "2025-06-21 15:14:42",
13     "updateTime": "2025-06-24 15:29:46"
14   },
15   "msg": "success",
16   "success": true
17 }
```

产品列表

- 接口地址: POST /api-service-product/api/app/product/list/{page}/{pageSize}
- 接口描述: 分页获取产品列表, 无需token

- 路径参数:

- page : 页码
- pageSize : 每页数量

- 请求参数:

代码块

```
1  {
```

```
2  "id": "Long",           // 产品ID 当传入产品ID时, 相当于调用queryOne
3  "productName": "String", // 产品名称 (可选)
4  "productImg": "String",  // 产品图片URL (可选)
5  "productDesc": "String", // 产品描述 (可选)
6  "price": "Integer",      // 产品单价 (可选)
7  "productNum": "Integer", // 库存数量 (可选)
8  "sortIndex": "Integer",  // 排序权重 (可选)
9  "enableFlag": "Boolean", // 是否启用 (可选true false)
10 "createTime": "String",  // 创建时间 (格式: yyyy-MM-dd HH:mm:ss, 可选)
11 "updateTime": "String"   // 更新时间 (格式同上, 可选)
12 }
```

- 请求示例:

代码块

```
1  {
2    "enableFlag": true
3  }
```

- 响应示例:

代码块

```
1  {
2    "code": 1,
3    "data": {
4      "records": [
5        {
6          "id": "1937411503942533120",
7          "productName": "哈密瓜",
8          "productImg": "",
9          "productDesc": "这是一个哈密瓜",
10         "price": 20,
11         "productNum": null,
12         "sortIndex": 0,
13         "enableFlag": true,
14         "createTime": "2025-06-24 15:24:32",
15         "updateTime": "2025-06-24 15:24:32"
16       },
17       {
18         "id": "1936321866654285824",
19         "productName": "雪梨",
20         "productImg": "",
21         "productDesc": "这是雪梨",
22         "price": 50,
```

```
23         "productNum": null,
24         "sortIndex": 0,
25         "enableFlag": true,
26         "createTime": "2025-06-21 15:14:42",
27         "updateTime": "2025-06-24 15:29:46"
28     }
29 ],
30     "total": "2",
31     "size": "6",
32     "current": "1",
33     "orders": [],
34     "optimizeCountSql": true,
35     "searchCount": true,
36     "maxLimit": null,
37     "countId": null,
38     "pages": "1"
39 },
40     "msg": "success",
41     "success": true
42 }
```

四、订单模块（service-order）

订单业务状态信息

代码块

```
1  CREATED(0, "已创建"),
2  PAID(1, "已支付"),
3  CANCELED(2, "已取消"),
4  //产品库存不够
5  INSUFFICIENT_STOCK(3, "库存不足结算失败"),
6  //用户余额不足
7  INSUFFICIENT_BALANCE(4, "余额不足结算失败"),
8  //"用户ID为空" 创建订单失败
9  USER_ID_EMPTY(5, "用户ID为空"),
10 //"商品ID为空" 创建订单失败
11 PRODUCT_ID_EMPTY(6, "商品ID为空"),
12 //购买数量 <=0 创建订单失败"
13 QUANTITY_INVALID(7, "购买数量<=0 创建订单失败"),
14 //"商品不存在" 创建订单失败
15 PRODUCT_NOT_EXIST(8, "商品不存在"),
16 //"用户不存在" 创建订单失败
17 USER_NOT_EXIST(9, "用户不存在"),
```



```
18 //没有admin权限
19 NO_ADMIN_ROLE(10,"没有admin权限"),
20 //查询结果为空
21 RESULT_EMPTY(11,"查询结果为空"),
22 //数据库操作失败
23 ORDER_FAILED(12,"数据库操作失败"),
24 //数据库操作成功
25 ORDER_SUCCESS(13,"数据库操作成功"),
26 //业务完成
27 COMPLETED (14,"业务完成"),
28 //业务失败
29 FAILED (15,"业务失败"),
30 //订单状态描述
31 //订单无法修改
32 CANNOT_MODIFY(16,"订单无法修改"),
33 //订单无法删除
34 CANNOT_DELETE(17,"订单无法删除"),
35 //订单无法修改数量
36 CANNOT_UPDATE_QUANTITY(18,"订单无法修改数量"),
37 //订单无法结算
38 CANNOT_SETTLE(19,"订单无法结算");
```

App接口（已测）

创建订单

- **接口地址:** POST /api-service-order/api/app/order/create
- **接口描述:** 创建订单，需要token
- **请求头:** token: {登录获取的token}
- **请求参数:**

代码块

```
1  {
2    "productId": "Long",    // 产品ID（必填）
3    "quantity": "Integer"   // 订单产品数量（必填）
4  }
```

- **响应示例:**

代码块

```
1  {
2    "code": 1,
```

```
3      "data": "1937419658057265152", // 订单id
4      "msg": "success",
5      "success": true
6  }
```

查询订单

- 接口地址: GET /api-service-order/api/app/order/queryByUserId
- 接口描述: 查询订单, 需要token
- 请求头: token: {登录获取的token}
- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": [
4          {
5              "id": "1937419658057265152", // 订单id
6              "productId": "1937411503942533120", // 产品id
7              "userId": "1937410285742366720", // 用户id
8              "quantity": 15, // 订单中产品的数量
9              "totalPrice": 300, // 本订单的总价
10             "status": "已创建", // 订单状态
11             "createTime": "2025-06-24 15:56:57",
12             "updateTime": "2025-06-24 15:56:57"
13         }
14     ],
15     "msg": "success",
16     "success": true
17 }
```

更新订单数量

- 接口地址: POST /api-service-order/api/app/order/updateQuantity
- 接口描述: 更新订单, 需要token
- 请求头: token: {登录获取的token}
- 请求参数:

代码块

```
1  {
2    "id": "Long",      // 订单ID (必填)
3  }
```

- 响应示例:

成功响应:

```
1  {
2    "code": 1,
3    "data": "业务完成",
4    "msg": "success",
5    "success": true
6  }
```

失败响应

```
1  {
2    "code": -1,
3    "data": null,
4    "msg": "余额不足结算失败订单无法结算",
5    "success": false
6  }
```

结算订单

- 接口地址: POST /api-service-order/api/app/order/settle
- 接口描述: 结算订单, 需要token
- 请求头: token: {登录获取的token}
- 请求参数:

代码块

```
1  {
2    "id": "Long",      // 订单ID (必填)
3  }
```

- 响应示例:

成功响应:

```
2      "code": 1,  
3      "data": "业务完成",  
4      "msg": "success",  
5      "success": true  
6  }
```

删除订单

- 接口地址: `POST /api-service-order/api/app/order/settle`
- 接口描述: 删除订单, 需要token
- 请求头: `token: {登录获取的token}`
- 请求参数:

代码块

```
1  {  
2      "id": "Long",      // 订单ID (必填)  
3  }
```

- 响应示例:

成功响应:

```
1  {  
2      "code": 1,  
3      "data": "业务完成",  
4      "msg": "success",  
5      "success": true  
6  }
```

Admin接口 (已测)

查询订单全仓

`GET /api/admin/order/queryAll`

只有admin 才有权限 获取全部订单 需要token (已经测试 当前版本没有实现admin登录 不再测试)

查询指定用户订单

GET `/api/admin/order/queryByUserId/{userId}`

五、推荐模块

- 接口地址: `POST /api-service-recommend/api/app/recommend/list`
- 接口描述: 获取3个产品数据, 需要用户token
- 请求头: `{token:"用户登录返回的token"}`
- 响应示例:

代码块

```
1  {
2      "code": 1,
3      "data": [
4          {
5              "id": "1937411503942533120",
6              "productName": "哈密瓜",
7              "productImg": "",
8              "productDesc": "这是一个哈密瓜",
9              "price": 20,
10             "productNum": 490,
11             "sortIndex": 0,
12             "enableFlag": true,
13             "createTime": "2025-06-24 15:24:32",
14             "updateTime": "2025-06-24 16:55:12"
15         },
16         {
17             "id": "1936321866654285824",
18             "productName": "雪梨",
19             "productImg": "",
20             "productDesc": "这是雪梨",
21             "price": 50,
22             "productNum": 500,
23             "sortIndex": 0,
24             "enableFlag": true,
25             "createTime": "2025-06-21 15:14:42",
26             "updateTime": "2025-06-24 16:54:52"
27         }
28     ],
29     "msg": "success",
30     "success": true
31 }
```

前端工程设计

代码块

```
1  frontend/
2  |—— index.html          # 首页（用于推荐商品）
3  |—— product-list.html   # 商品列表页（分页展示所有商品）
4  |—— product-detail.html # 商品详情页（展示单个商品的详细数据）
5  |—— user.html           # 用户中心页（需要先登录，登录后可以看到个人信息以及自己的订
    单）
6  |—— style.css           # 统一样式文件
7  |—— SWAPI.md            # 接口文档（用来喂给ai跑代码）
8  |—— api.js              # API接口封装
```

项目打包及部署

项目打包

代码块

```
1  ----- 单模块打包
2  mvn clean package -pl gateway -am "-Dmaven.test.skip=true"
3  mvn clean package -pl services/service-user -am "-Dmaven.test.skip=true"
4  mvn clean package -pl services/service-product -am "-Dmaven.test.skip=true"
5  mvn clean package -pl services/service-order -am "-Dmaven.test.skip=true"
6  mvn clean package -pl services/service-recommend -am "-Dmaven.test.skip=true"
7
8  ----- 全项目打包
9  mvn clean package -pl gateway,services/service-user,services/service-
    product,services/service-order,services/service-recommend -am "-
    Dmaven.test.skip=true"
```

中间件部署

中间件

nacos

[nacos 官网](#)

windows 下载zip 文件后,解压,

进入bin 目录下,执行 `startup.cmd -m standalone`,以单机模式启动(内置的数据库Derby)

如果提示没有设置JAVA_HOME,请在系统环境变量中设置JAVA_HOME环境变量

例如 `JAVA_HOME= C:\Program Files\Java\jdk1.8.0_191`

然后在Path中添加%JAVA_HOME%\bin

启动后,进入nacos管理页面:<http://192.168.31.11:8848/nacos/index.html> (根据本机ip变化)

或者 <http://localhost:8848/nacos/index.html>

注意:现在是使用单机模式, 正式环境需要集群模式,把数据存在mysql数据库中

redis

1. 安装docker

2. 拉取redis镜像

3. `docker run -d --name redis -p 6379:6379 redis:5.0`

nginx

1. 安装nginx

2. `cd /etc/nginx/conf.d`

3. 测试配置文件语法

`nginx -t`

4. 重启Nginx生效

`systemctl restart nginx`

各服务启动

打包后,找到jar文件,使用java -jar xxx.jar启动服务

代码块

```
1  # 启动product
2  nohup java -jar service-product-1.0-SNAPSHOT.jar --server.port=9028 --
    spring.profile.active=dev > service-product.log 2>&1 &
3  #nohup java -jar service-product-1.0-SNAPSHOT.jar --server.port=9029 --
    spring.profile.active=dev > service-product1.log 2>&1 &
```

```
4
5 # 启动user服务
6 nohup java -jar service-user-1.0-SNAPSHOT.jar --server.port=9030 --
  spring.profile.active=dev > service-user.log 2>&1 &
7 #nohup java -jar service-user-1.0-SNAPSHOT.jar --server.port=9031 --
  spring.profile.active=dev > service-user1.log 2>&1 &
8 #nohup java -jar service-user-1.0-SNAPSHOT.jar --server.port=9032 --
  spring.profile.active=dev > service-user2.log 2>&1 &
9
10 # 启动order服务
11 nohup java -jar service-order-1.0-SNAPSHOT.jar --server.port=9033 --
  spring.profile.active=dev > service-order.log 2>&1 &
12 #nohup java -jar service-order-1.0-SNAPSHOT.jar --server.port=9034 --
  spring.profile.active=dev > service-order1.log 2>&1 &
13 #nohup java -jar service-order-1.0-SNAPSHOT.jar --server.port=9035 --
  spring.profile.active=dev > service-order2.log 2>&1 &
14
15 # 启动recommend服务
16 nohup java -jar service-recommend-1.0-SNAPSHOT.jar --server.port=9036 --
  spring.profile.active=dev > service-recommend.log 2>&1 &
17
18 # 启动网关
19 nohup java -jar gateway-1.0-SNAPSHOT.jar --server.port=9025 --
  spring.profile.active=dev > gateway.log 2>&1 &
```

- 服务停止

代码块

```
1 pkill -f 'service-product-1.0-SNAPSHOT.jar'
2 pkill -f 'service-user-1.0-SNAPSHOT.jar'
3 pkill -f 'service-order-1.0-SNAPSHOT.jar'
4 pkill -f 'service-recommend-1.0-SNAPSHOT.jar'
5 pkill -f 'gateway-1.0-SNAPSHOT.jar'
```